

C++ STL Internals: vector, deque, priority_queue, set/map, unordered_map

1. Overview

STL containers are ready-made data structures in C++ Standard Template Library. In interviews, you are often expected to know not only how to use them, but also what happens internally.

Definition:

This topic covers how common STL containers are implemented internally:

STL Container	Internal Data Structure
<code>vector</code>	Dynamic array
<code>deque</code>	Block array / segmented array
<code>priority_queue</code>	Binary heap, usually max-heap
<code>set, map</code>	Self-balancing BST, usually red-black tree
<code>unordered_map</code>	Hash table

Why it matters:

- Helps choose the right container in coding rounds.
- Explains time complexity beyond memorization.
- Helps avoid performance bugs like repeated vector reallocations.
- Useful for online assessments where constraints decide the data structure.

Where it is used in real systems:

- `vector`: storing dynamic lists, adjacency lists, buffers.
- `deque`: task queues, sliding window problems, double-ended operations.
- `priority_queue`: schedulers, shortest path algorithms, event processing.
- `set/map`: sorted dictionaries, range queries, ordered indexes.
- `unordered_map`: fast lookup caches, frequency maps, symbol tables.

Why interviewers ask about it:

Interviewers want to check whether you understand the trade-off between speed, memory, ordering, iterator validity, and worst-case behavior. Good candidates do not just say "`unordered_map` is $O(1)$ "; they know it can degrade and why.

2. Core Idea

The core idea is that STL containers give a clean interface, but each interface is powered by a specific internal data structure.

Intuition

Think of STL containers like different storage systems:

Need	Best Mental Model	STL Container
Store items compactly and access by index	Expandable shelf	<code>vector</code>
Push/pop at both ends	Linked blocks of shelves	<code>deque</code>
Always get largest/smallest quickly	Tournament tree	<code>priority_queue</code>
Keep keys sorted	Balanced search tree	<code>set</code> , <code>map</code>
Find by key very fast	Labeled buckets	<code>unordered_map</code>

Real-world analogy

Suppose you manage books:

- `vector`: books are placed continuously on one long shelf.
- `deque`: books are stored in multiple shelf blocks, with a directory pointing to each block.
- `priority_queue`: books are arranged so the most urgent one is always on top.
- `set/map`: books are arranged alphabetically in a balanced tree-like catalog.
- `unordered_map`: books are placed into buckets based on a hash of the title.

Small example

```
vector<int> v = {10, 20, 30};
v.push_back(40);

priority_queue<int> pq;
pq.push(5);
pq.push(20);
pq.push(10);
cout << pq.top(); // 20

unordered_map<string, int> freq;
freq["cpp"]++;
```

Step-by-step explanation

1. You choose an STL container based on operation needs.
2. The container maps those operations to an internal structure.
3. The internal structure decides time complexity.
4. Memory layout affects cache performance and iterator invalidation.
5. Interview answers should include both complexity and trade-offs.

3. Important Subtopics

3.1 `vector`: Dynamic Array

What it means:

vector stores elements in contiguous memory. When capacity is full, it allocates a bigger array, moves/copies old elements, deletes old storage, and inserts the new element.

Why it matters:

Contiguous memory gives fast indexing and excellent cache locality. But insertion/deletion in the middle is costly.

Example:

```
vector<int> v;  
v.push_back(1);  
v.push_back(2);  
v.push_back(3);  
cout << v[1]; // O(1), prints 2
```

Common interview angle:

Why is **push_back** amortized $O(1)$, not always $O(1)$?

Key points:

- **size()** = number of elements.
- **capacity()** = allocated storage.
- Reallocation invalidates iterators, pointers, and references.
- Random access is $O(1)$.
- Middle insertion/deletion is $O(n)$.

3.2 **deque**: Block Array / Segmented Array

What it means:

deque stands for double-ended queue. Internally, it is usually implemented as multiple fixed-size blocks, with a central map/directory pointing to those blocks.

Why it matters:

It supports efficient push and pop at both front and back.

Example:

```
deque<int> dq;  
dq.push_back(10);  
dq.push_front(5);  
dq.pop_back();
```

Common interview angle:

How is **deque** different from **vector** if both support random access?

Key points:

- Random access is $O(1)$, but less cache-friendly than `vector`.
- Push/pop front and back are $O(1)$ amortized.
- Not stored in one contiguous memory block.
- Middle insertion/deletion is still $O(n)$.

3.3 `priority_queue`: Heap

What it means:

`priority_queue` is an adapter, not a standalone raw container. By default, it uses a `vector` internally and maintains heap order.

Why it matters:

It gives fast access to the highest-priority element.

Example:

```
priority_queue<int> pq;
pq.push(30);
pq.push(10);
pq.push(50);
cout << pq.top(); // 50
```

Common interview angle:

Why are push and pop $O(\log n)$, but top is $O(1)$?

Key points:

- Default is max-heap.
- `top()` returns max element by default.
- `push()` performs heapify-up.
- `pop()` swaps root with last, removes last, then heapify-down.
- Does not support searching or deleting arbitrary elements efficiently.

3.4 `set` and `map`: Red-Black Tree

What it means:

`set` stores unique sorted keys. `map` stores sorted key-value pairs. They are commonly implemented using red-black trees, a type of self-balancing binary search tree.

Why it matters:

Operations remain $O(\log n)$ even when data is inserted in sorted order.

Example:

```
set<int> s = {5, 1, 3};
// Stored in sorted order: 1, 3, 5
```

```
map<string, int> marks;  
marks["Asha"] = 95;
```

Common interview angle:

Why use `map` instead of `unordered_map`?

Key points:

- Maintains sorted order.
- Search, insert, delete are $O(\log n)$.
- Supports `lower_bound`, `upper_bound`, range queries.
- Keys must be comparable using `<` or custom comparator.

3.5 Red-Black Tree

What it means:

A red-black tree is a balanced BST where each node has a color, red or black. Color rules keep the tree height approximately logarithmic.

Why it matters:

Without balancing, a BST can become a linked list and operations can degrade to $O(n)$.

Example:

If you insert:

```
1, 2, 3, 4, 5
```

A normal BST may become:



A red-black tree rotates and recolors nodes to stay balanced.

Common interview angle:

Do you need to know exact red-black insertion cases? Usually not for placement interviews, but you should know why balancing is needed.

Key points:

- Root is black.
- Red nodes cannot have red children.
- Every root-to-leaf path has the same number of black nodes.
- Rotations and recoloring restore balance.
- Height is $O(\log n)$.

3.6 `unordered_map`: Hash Table

What it means:

`unordered_map` stores key-value pairs using hashing. A hash function converts a key into a bucket index.

Why it matters:

Average lookup, insertion, and deletion are $O(1)$, making it very useful in coding problems.

Example:

```
unordered_map<string, int> freq;
freq["apple"]++;
freq["banana"]++;
```

Common interview angle:

Why can `unordered_map` become $O(n)$?

Key points:

- Uses hash function + bucket array.
- Collisions happen when different keys go to the same bucket.
- Average case is $O(1)$.
- Worst case is $O(n)$, especially with many collisions.
- Rehashing occurs when load factor grows too high.

4. Real-World Example

Backend server request handling

Imagine a backend service handling user requests:

Requirement	Container
Store active user IDs compactly	<code>vector<int></code>
Maintain recent requests from both ends	<code>deque<Request></code>
Process most urgent jobs first	<code>priority_queue<Job></code>
Keep users sorted by username	<code>map<string, User></code>
Fast lookup by session token	<code>unordered_map<string, Session></code>

Example:

```
unordered_map<string, Session> sessions;
priority_queue<Job> urgentJobs;
deque<Request> recentRequests;
map<string, User> usersByName;
```

In real systems, choosing the wrong container can cause performance problems. For example, using `map` for millions of random exact lookups may be slower than `unordered_map`; using `vector` for frequent front insertions may be inefficient.

5. Diagrams / Mental Models

`vector` memory model

Contiguous memory:

```
Index:   0     1     2     3
        +----+----+----+----+
Value: | 10 | 20 | 30 | 40 |
        +----+----+----+----+

size = 4
capacity may be 4, 6, 8, etc.
```

When full:

```
Old array: [10][20][30][40]

Allocate bigger array:
New array: [10][20][30][40][50][ ][ ][ ]
```

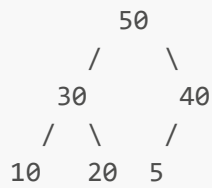
`deque` memory model

Directory / map of blocks:

```
Map ----> +-----+-----+-----+
           | Block0 | Block1 | Block2 |
           +-----+-----+-----+
           |       |       |       |
           v       v       v
        [1 2 3]  [4 5 6]  [7 8 9]
```

Heap mental model for `priority_queue`

Max heap:



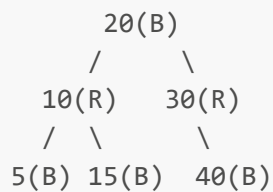
Array representation:

[50, 30, 40, 10, 20, 5]

For index i :

Relationship	Formula
Parent	$(i - 1) / 2$
Left child	$2 * i + 1$
Right child	$2 * i + 2$

Red-black tree mental model



The exact shape may differ, but the important idea is balanced height.

Hash table mental model

```
unordered_map<string, int>
```

```
hash("cat") % bucket_count = 2
```

```
hash("dog") % bucket_count = 0
```

```
Bucket 0: ("dog", 1)
```

```
Bucket 1:
```

```
Bucket 2: ("cat", 1) -> ("act", 1) collision chain
```

```
Bucket 3:
```

6. Common Interview Questions

Q1. How does **vector** work internally?

Answer:

`vector` uses a dynamic contiguous array. It keeps track of size and capacity. When capacity is full, it allocates a larger array, moves/copies existing elements, and frees the old array.

Expected key points:

- Contiguous memory.
- $O(1)$ indexing.
- Amortized $O(1)$ `push_back`.
- Reallocation can invalidate iterators.

Common mistakes:

- Saying `vector` is a linked list.
- Saying every `push_back` is always $O(1)$.

Q2. Why is `vector::push_back` amortized $O(1)$?

Answer:

Most insertions happen directly if capacity is available. Occasionally, reallocation costs $O(n)$. Over many insertions, the total cost averages to $O(1)$ per insertion.

Expected key points:

- Capacity grows geometrically.
- Rare expensive reallocations.
- Average over sequence of operations.

Common mistakes:

- Ignoring reallocation.
- Confusing average case with amortized analysis.

Q3. What is the difference between `size()` and `capacity()` in `vector`?

Answer:

`size()` is the number of actual elements. `capacity()` is the number of elements that can fit before reallocation is needed.

Expected key points:

- `size <= capacity`.
- `reserve()` changes capacity.
- `resize()` changes size.

Common mistakes:

- Thinking `reserve()` inserts elements.
- Thinking capacity always equals size.

Q4. How is `deque` different from `vector`?

Answer:

`vector` stores elements contiguously. `deque` stores elements in multiple blocks and supports efficient insertion/removal at both front and back.

Expected key points:

- `vector`: better cache locality.
- `deque`: $O(1)$ push/pop front and back.
- Both support random access.

Common mistakes:

- Saying `deque` is a doubly linked list.
- Saying `deque` has no random access.

Q5. How does `priority_queue` work internally?

Answer:

It uses a heap, usually stored in a `vector`. The largest element is kept at the root in default max-heap behavior.

Expected key points:

- Default max-heap.
- `top()` is $O(1)$.
- `push()` and `pop()` are $O(\log n)$.
- Implemented as container adapter.

Common mistakes:

- Saying it stores all elements in sorted order.
- Expecting efficient search inside `priority_queue`.

Q6. How do you create a min-heap using `priority_queue`?

Answer:

```
priority_queue<int, vector<int>, greater<int>> minHeap;
```

Expected key points:

- Third template argument is comparator.
- `greater<int>` creates min-heap behavior.

Common mistakes:

- Reversing comparator logic.
- Forgetting to include `<queue>` and `<functional>`.

Q7. How are `set` and `map` implemented internally?

Answer:

They are usually implemented using red-black trees. This keeps keys sorted and gives $O(\log n)$ insertion, deletion, and search.

Expected key points:

- Ordered containers.
- Balanced BST.
- Support range operations.

Common mistakes:

- Saying they use hash tables.
- Saying lookup is $O(1)$.

Q8. What is the difference between `map` and `unordered_map`?

Answer:

`map` stores keys in sorted order using a balanced tree and gives $O(\log n)$ operations. `unordered_map` uses hashing and gives average $O(1)$ operations but does not maintain order.

Expected key points:

- Ordering vs hashing.
- $O(\log n)$ vs average $O(1)$.
- Worst-case behavior of `unordered_map`.

Common mistakes:

- Saying `unordered_map` is always faster.
- Ignoring ordered traversal and range queries.

Q9. What are hash collisions?

Answer:

A collision occurs when two different keys map to the same bucket in a hash table.

Expected key points:

- Collisions are normal.
- Handled by chaining or open addressing depending on implementation.
- Too many collisions hurt performance.

Common mistakes:

- Thinking hash functions must never collide.
- Thinking collision means data is lost.

Q10. What is load factor in `unordered_map`?

Answer:

Load factor is the ratio of number of elements to number of buckets.

```
load_factor = size / bucket_count
```

When it becomes too high, the table may rehash to increase buckets.

Expected key points:

- High load factor means more collisions.
- Rehashing is expensive.
- `reserve()` can reduce rehashing.

Common mistakes:

- Confusing load factor with time complexity.
- Forgetting rehash invalidates iterators.

Q11. When should you prefer `set` over `unordered_set`?

Answer:

Use `set` when you need sorted order, range queries, `lower_bound`, `upper_bound`, or predictable $O(\log n)$ worst-case operations.

Expected key points:

- Sorted traversal.
- Range operations.
- Worst-case guarantee.

Common mistakes:

- Choosing only based on average lookup speed.

Q12. Which STL containers invalidate iterators on insertion?

Answer:

It depends on the container. `vector` insertion may invalidate all iterators if reallocation occurs. `map` and `set` generally preserve iterators to existing elements after insertion. `unordered_map` insertion may invalidate iterators if rehashing occurs.

Expected key points:

- `vector`: reallocation danger.
- `deque`: more complex invalidation rules.
- `map/set`: stable node-based iterators.
- `unordered_map`: rehash invalidates iterators.

Common mistakes:

- Giving one rule for all containers.

7. Deep-Dive Questions

Q1. Why does **vector** usually grow by a factor instead of increasing capacity by 1?

If capacity increased by 1 each time, inserting n elements would require repeated copying and become $O(n^2)$. Geometric growth keeps total copying linear over many insertions, making **push_back** amortized $O(1)$.

Q2. Why is **deque** random access $O(1)$ even though memory is not contiguous?

The implementation can compute which block contains the element and the offset inside that block.

```
index -> block number + offset inside block
```

This takes constant time, but with slightly more overhead than **vector**.

Q3. Why does **priority_queue** not allow iteration in sorted order?

Because a heap is only partially ordered. It guarantees the root is highest priority, but it does not fully sort siblings or subtrees. To get sorted order, you must repeatedly pop elements, costing $O(n \log n)$.

Q4. Why are red-black trees used instead of normal BSTs?

A normal BST can become skewed if input is sorted, making operations $O(n)$. Red-black trees maintain balance using color rules, rotations, and recoloring, keeping operations $O(\log n)$.

Q5. How can **unordered_map** be attacked or slowed down?

If many keys collide into the same bucket, operations can degrade toward $O(n)$. Poor custom hash functions, adversarial inputs, or high load factor can cause this. In competitive programming, custom hashes are sometimes used to avoid collision-heavy hacks.

8. Comparison Tables

vector vs **deque**

Feature	vector	deque
Internal structure	Dynamic contiguous array	Multiple fixed-size blocks
Random access	$O(1)$	$O(1)$
Push back	Amortized $O(1)$	$O(1)$ amortized
Push front	$O(n)$	$O(1)$ amortized
Cache locality	Excellent	Good, but weaker than vector
Memory contiguous	Yes	No
Best use	Dynamic array, indexing	Queue-like access at both ends

map vs **unordered_map**

Feature	map	unordered_map
Internal structure	Red-black tree	Hash table
Ordering	Sorted by key	No ordering
Search	$O(\log n)$	Average $O(1)$, worst $O(n)$
Insert/delete	$O(\log n)$	Average $O(1)$, worst $O(n)$
Range queries	Efficient	Not supported directly
Key requirement	Comparable	Hashable and equality comparable
Use when	Need order/ranges	Need fast exact lookup

set vs multiset

Feature	set	multiset
Duplicate keys	Not allowed	Allowed
Internal structure	Red-black tree	Red-black tree
Search	$O(\log n)$	$O(\log n)$
Common use	Unique sorted values	Sorted values with duplicates

priority_queue vs set

Feature	priority_queue	set
Main operation	Get max/min quickly	Maintain sorted unique values
Internal structure	Heap	Red-black tree
Top/min/max access	$O(1)$ for top	$O(1)$ for begin/rbegin access
Insert	$O(\log n)$	$O(\log n)$
Delete arbitrary element	Not efficient	$O(\log n)$
Search	Not efficient	$O(\log n)$
Sorted iteration	No	Yes

Ordered vs Unordered Containers

Category	Ordered	Unordered
Examples	set, map	unordered_set, unordered_map
Internal data structure	Balanced BST	Hash table
Maintains sorted order	Yes	No
Average lookup	$O(\log n)$	$O(1)$

Category	Ordered	Unordered
Worst-case lookup	$O(\log n)$	$O(n)$
Range queries	Yes	No

9. Common Mistakes

- Saying `vector` is always $O(1)$ for insertion.
- Forgetting that `vector` reallocation invalidates pointers and iterators.
- Thinking `deque` is implemented as a doubly linked list.
- Thinking `priority_queue` stores elements in fully sorted order.
- Using `priority_queue` when arbitrary deletion is needed.
- Saying `map` uses hashing.
- Saying `unordered_map` is always $O(1)$.
- Ignoring hash collisions.
- Forgetting that `map` sorts by key, not by value.
- Confusing `reserve()` and `resize()`.
- Using `unordered_map` when sorted output is required.
- Using `vector.erase(v.begin())` repeatedly, causing $O(n^2)$ behavior.

10. Edge Cases / Special Cases

`vector`

- Reallocation invalidates all iterators, references, and pointers to elements.
- `reserve(n)` allocates capacity but does not create elements.
- `resize(n)` changes the number of elements.
- `vector<bool>` is a special optimized representation and does not behave exactly like normal `vector<T>`.

`deque`

- Not contiguous, so it cannot be passed as a raw array.
- Random access is $O(1)$, but with extra indirection.
- Iterator invalidation rules are more complex than `vector`.

`priority_queue`

- No direct way to decrease key, unlike some theoretical heap APIs.
- No efficient arbitrary deletion.
- Custom comparator can be confusing because it defines priority ordering.

`set/map`

- Keys in `set` and keys in `map` are effectively immutable because changing a key can break tree ordering.
- Duplicate keys require `multiset` or `multimap`.
- Custom comparator must define strict weak ordering.

`unordered_map`

- Worst-case operations can become $O(n)$.
- Rehashing is expensive and invalidates iterators.
- Custom keys need custom hash and equality.
- Output order is not predictable.

11. How to Explain in Interview

"C++ STL containers are built on different internal data structures. `vector` is a dynamic contiguous array, so indexing is $O(1)$ and push back is amortized $O(1)$, but middle insertion is $O(n)$. `deque` uses multiple blocks, so it supports efficient front and back operations. `priority_queue` uses a heap, giving $O(1)$ top and $O(\log n)$ push/pop. `set` and `map` are usually red-black trees, so they keep keys sorted with $O(\log n)$ operations. `unordered_map` uses a hash table, so lookup is average $O(1)$, but collisions can make it worse."

12. Quick Revision Notes

Key definitions

- `vector`: dynamic contiguous array.
- `deque`: double-ended queue using segmented blocks.
- `priority_queue`: heap-based container adapter.
- `set`: sorted unique keys.
- `map`: sorted key-value pairs.
- Red-black tree: self-balancing BST.
- `unordered_map`: hash table-based key-value store.

Important points

- `vector` has best cache locality.
- `deque` is good for push/pop at both ends.
- `priority_queue` is not fully sorted.
- `map/set` maintain order.
- `unordered_map` is fastest on average for exact lookup.
- Hash collisions affect `unordered_map` performance.

Common comparisons

- `vector` vs `deque`: contiguous array vs block array.
- `map` vs `unordered_map`: sorted tree vs hash table.
- `priority_queue` vs `set`: heap top access vs full sorted structure.
- `set` vs `multiset`: unique vs duplicate sorted keys.

Must-remember facts

Container	Search	Insert	Delete	Maintains Order
<code>vector</code>	$O(n)$, $O(1)$ by index	End amortized $O(1)$, middle $O(n)$	$O(n)$	In insertion order

Container	Search	Insert	Delete	Maintains Order
<code>deque</code>	$O(n)$, $O(1)$ by index	Ends $O(1)$, middle $O(n)$	Ends $O(1)$, middle $O(n)$	In insertion order
<code>priority_queue</code>	$O(n)$	$O(\log n)$	Top $O(\log n)$	Heap order only
<code>set</code>	$O(\log n)$	$O(\log n)$	$O(\log n)$	Sorted
<code>map</code>	$O(\log n)$	$O(\log n)$	$O(\log n)$	Sorted by key
<code>unordered_map</code>	Average $O(1)$	Average $O(1)$	Average $O(1)$	No

Interview traps

- `unordered_map` is not always $O(1)$.
- `priority_queue` is not a sorted array.
- `deque` is not a linked list.
- `vector::reserve()` does not change size.
- `map` orders by key, not insertion order.

13. Practice Tasks

Task 1: Trace vector growth

Write a program that prints `size()` and `capacity()` after every `push_back`.

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    vector<int> v;
    for (int i = 0; i < 20; i++) {
        v.push_back(i);
        cout << "size=" << v.size()
              << " capacity=" << v.capacity() << '\n';
    }
}
```

Task 2: Sliding window maximum

Solve sliding window maximum using `deque`. This teaches why `deque` is useful for front and back operations.

Task 3: Implement heap operations

Implement `push_heap` and `pop_heap` logic manually on a `vector<int>` to understand `priority_queue`.

Task 4: Compare `map` and `unordered_map`

Count word frequencies using both containers. Print output and observe that `map` gives sorted keys while `unordered_map` does not.

Task 5: Use `lower_bound` in `set`

Given a set of numbers, find the smallest number greater than or equal to x.

```
set<int> s = {10, 20, 30, 40};
auto it = s.lower_bound(25); // points to 30
```

Task 6: Custom comparator in `priority_queue`

Create a min-heap of pairs based on the second value.

Task 7: Custom hash key

Use `unordered_map<pair<int, int>, int>` by writing a custom hash function.

Task 8: Choose the right container

For each requirement, choose the best STL container:

Requirement	Suggested Container
Need sorted unique numbers	<code>set</code>
Need fast frequency count	<code>unordered_map</code>
Need smallest element repeatedly	<code>priority_queue</code> as min-heap
Need frequent push front and push back	<code>deque</code>
Need compact index-based storage	<code>vector</code>

14. Final Cheat Sheet

Core definition

C++ STL containers are high-level data structures backed by specific internals: dynamic arrays, block arrays, heaps, red-black trees, and hash tables.

Why it matters

Knowing internals helps you choose the correct container, predict time complexity, avoid iterator bugs, and explain trade-offs in interviews.

Most asked questions

- How does `vector` grow?
- Why is `push_back` amortized $O(1)$?
- Difference between `vector` and `deque`?

- How does `priority_queue` use a heap?
- Difference between `map` and `unordered_map`?
- Why is `set/map` $O(\log n)$?
- What are hash collisions?
- When can `unordered_map` become $O(n)$?
- What is load factor?
- Which operations invalidate iterators?

Common comparisons

Comparison	Main Difference
<code>vector</code> vs <code>deque</code>	Contiguous dynamic array vs segmented block array
<code>map</code> vs <code>unordered_map</code>	Sorted red-black tree vs hash table
<code>set</code> vs <code>unordered_set</code>	Sorted $O(\log n)$ vs average $O(1)$ lookup
<code>priority_queue</code> vs <code>set</code>	Fast top element vs searchable sorted structure
<code>reserve</code> vs <code>resize</code>	Capacity allocation vs actual element count

One-line interview answer

"`vector` is a dynamic array, `deque` is a segmented block array, `priority_queue` is a heap, `set/map` are usually red-black trees, and `unordered_map` is a hash table; their internal structures decide ordering, complexity, memory behavior, and iterator validity."